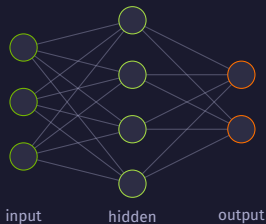


Module 02

Deep Learning Fundamentals

Neural Networks dengan TensorFlow



Act 1

Neural Network Pertama

Membangun jaringan saraf pertama dengan TensorFlow

Apa itu Neural Network?

Meniru cara otak memproses informasi

- ▶ **Neuron:** unit pemrosesan terkecil, seperti sel saraf
- ▶ **Layer:** kumpulan neuron yang bekerja bersama
- ▶ **Weights** (bobot): kekuatan koneksi yang dipelajari
- ▶ Alur: **input** → **hidden layers** → **output** (prediksi)



NN belajar pola dengan menyetel ribuan **weights**, bukan aturan manual. Analogi koki: weight = takaran bumbu, training = coba sampai pas.

Dataset & Normalisasi

Fashion MNIST: 70.000 gambar pakaian

- ▶ **Fashion MNIST:** 28×28 grayscale, 10 kelas
- ▶ 60.000 train + 10.000 test; 5.000 validasi dipisah dari train
- ▶ Pixel asli 0–255 $\rightarrow$ normalisasi (bagi 255) jadi 0.0–1.0
- ▶ Manfaat: training stabil & cepat, **gradient** terkendali

```
1 X_train = X_train / 255.0
2 X_test  = X_test  / 255.0
```

Aspek	Nilai
Ukuran gambar	$28 \times 28 = 784$ pixel
Jumlah kelas	10 (T-shirt . . . Boot)
Pixel sebelum	0–255 (uint8)
Pixel sesudah	0.0–1.0 (float)

Membangun Model

Sequential: Flatten menuju Dense bertingkat

- ▶ **Sequential:** tumpukan layer input → output
- ▶ **Flatten:** 28×28 jadi 784 angka
- ▶ **Dense(128)** lalu **Dense(64)**, **ReLU**
- ▶ **Dense(10) Softmax:** jadi probabilitas
- ▶ Total ~109K **parameter** dipelajari

```
1 model = tf.keras.Sequential([  
2     layers.Flatten(input_shape=(28,28)),  
3     layers.Dense(128, activation='relu'),  
4     layers.Dense(64,  activation='relu'),  
5     layers.Dense(10,  activation='softmax'  
6 ])
```

Tiap parameter = satu nilai yang dipelajari; training mengoptimalkan semuanya sekaligus.

Training & Loss

compile lalu fit selama 20 epoch

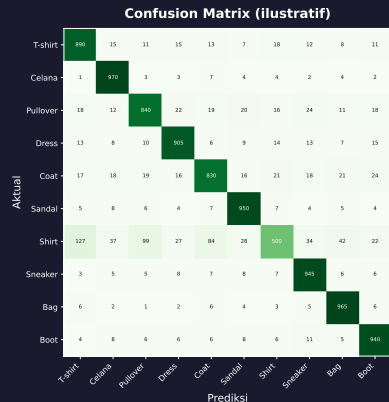
- ▶ **compile**: butuh **loss, optimizer, metrics**
- ▶ **Loss** (crossentropy): ukur seberapa salah
- ▶ **Optimizer** Adam: perbaiki weights tiap langkah
- ▶ **Epoch**: 1 putaran lihat semua data (per **batch**)
- ▶ Harapan: loss turun, train & val rapat (hindari **overfitting**)

```
1 model.compile(  
2     loss='  
        sparse_categorical_crossentropy'  
3     ,  
4     optimizer='adam',  
5     metrics=['accuracy'])  
6 history = model.fit(X_train, y_train,  
7                     epochs=20, batch_size=32,  
8                     validation_data=(X_valid, y_valid))
```

Evaluasi Model

Ujian akhir dengan data yang belum pernah dilihat

- ▶ Test set: tipikal $\sim 88\%$ accuracy (per run berbeda)
- ▶ **Confusion matrix:** kelas mana sering tertukar
- ▶ Shirt vs T-shirt sering tertukar (mirip visual)
- ▶ Softmax beri probabilitas untuk tiap dari 10 kelas
- ▶ Prediksi = kelas dengan probabilitas tertinggi



Act 2

Aktivasi & Optimizer

Membuat jaringan belajar lebih baik

Kenapa Perlu Aktivasi?

Tanpa non-linearity, jaringan tetap satu garis lurus

- ▶ Tanpa **activation**, NN cuma jumlah & kali
- ▶ Banyak layer linear ditumpuk = tetap 1 fungsi linear
- ▶ **Activation** tambah **non-linearity** → kenali pola berkelok
- ▶ Inti pengenalan gambar, teks, dan suara

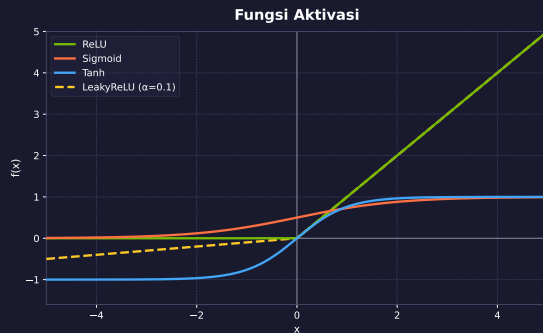
```
1 layer2      = W2 @ (W1 @ x)
2 W_gabungan = W2 @ W1
3 satu_layer = W_gabungan @ x
4 # layer2 == satu_layer (sama!)
```

Tanpa activation, 100 layer = 1 layer linear belaka.

Fungsi Aktivasi Utama

Bentuk dan kegunaan tiap fungsi

- ▶ **ReLU**: $\max(0, x)$; cepat, standar industri
- ▶ **Sigmoid**: 0–1, cocok probabilitas
- ▶ **Tanh**: -1–1, zero-centered, stabil
- ▶ **LeakyReLU**: gradient kecil di sisi negatif
- ▶ Risiko: **vanishing gradient**, **dead neuron**



ReLU mendominasi hidden layer karena cepat dan sederhana.

Softmax untuk Klasifikasi

Mengubah logits menjadi probabilitas

- ▶ Bekerja pada seluruh output layer sekaligus
- ▶ Ubah skor mentah (**logits**) jadi probabilitas
- ▶ Semua probabilitas selalu berjumlah 100%
- ▶ Nilai tertinggi diperkuat eksponensial
- ▶ Dipakai di output layer multi-kelas



angka bawah = **logit**; tinggi batang = **probabilitas** (softmax, jumlah = 1)

Softmax = penerjemah skor mentah ke probabilitas multi-kelas.

Panduan Memilih Aktivasi

Aturan praktis sesuai jenis layer

Situasi	Aktivasi	Alasan
Hidden layer (default)	ReLU	Cepat, efektif, standar
Hidden (advanced)	Swish / LeakyReLU	Performa sedikit lebih baik
Output biner	Sigmoid	Probabilitas Ya/Tidak (0-1)
Output multi-kelas	Softmax	Probabilitas tiap kelas
Output regresi	Linear (None)	Prediksi angka bebas

Default aman: ReLU di hidden, Sigmoid biner, Softmax multiclass.

Optimizer

Bagaimana model memperbaiki kesalahannya

- ▶ **Optimizer:** cara update **weights** via **gradient**
- ▶ Analogi: turun gunung berkabut cari titik terendah
- ▶ **SGD**+Momentum: langkah kecil, inersia stabil
- ▶ **RMSprop:** sesuaikan langkah; cocok RNN/LSTM
- ▶ **Adam:** adaptif arah + kecepatan; paling populer

```
1 model.compile(  
2     loss='sparse_categorical'  
3         '_crossentropy',  
4     optimizer='adam',  
5     metrics=['accuracy'])
```

Mulai selalu dengan Adam; pindah SGD+Momentum bila tak stabil.

Act 3

CNN & Transfer Learning

Neural network untuk gambar

Apa itu CNN?

Convolutional Neural Network: cara komputer melihat gambar



- ▶ **Convolution:** filter kecil memindai gambar cari pola lokal
- ▶ **Filter/kernel** 3×3 : deteksi tepi, garis, tekstur

- ▶ **Pooling:** menyusutkan ukuran, simpan fitur penting
- ▶ **Weights** di-share $\rightarrow$ parameter jauh lebih sedikit

Membangun CNN dari Nol

Conv2D + MaxPooling2D + Dense pada CIFAR-10

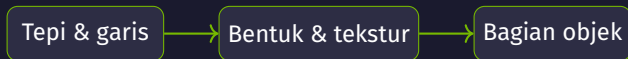
- ▶ **CIFAR-10**: 60K gambar warna 32×32 , 10 kelas
- ▶ Blok Conv2D (**ReLU**) $\rightarrow$ MaxPooling2D
- ▶ Tiap pooling: dimensi $32 \rightarrow 16 \rightarrow 8 \rightarrow 4$, filter bertambah
- ▶ Flatten $\rightarrow$ Dense $\rightarrow$ Dropout $\rightarrow$ Dense(10)

```
1 model = Sequential([
2     Conv2D(32,(3,3),activation='relu',padding='same',
3         input_shape=(32,32,3)),
4     MaxPooling2D(), # 32->16
5     Conv2D(64,(3,3),activation='relu',padding='same'),
6     MaxPooling2D(), # 16->8
7     Conv2D(128,(3,3),activation='relu',padding='same'),
8     MaxPooling2D(), # 8->4
9     Flatten(), Dense(128,'relu'),
10    Dropout(0.5), Dense(10,'softmax')
11 ])
```

Apa yang Dipelajari CNN?

Feature maps: dari tepi sampai objek

- ▶ **Feature maps:** hasil tiap filter saat memproses gambar
- ▶ Layer awal: tepi, garis, warna sederhana
- ▶ Layer tengah: gabung jadi bentuk & tekstur
- ▶ Layer akhir: bagian objek (telinga, roda, sayap)



CNN membangun pemahaman bertingkat secara otomatis: tepi → bentuk → objek.

Transfer Learning

Pakai ResNet50 yang sudah pintar

- ▶ Analogi: fasih B.Indonesia → belajar B.Melayu lebih cepat
- ▶ **ResNet50** pretrained di ImageNet (14 juta gambar)
- ▶ **Freeze** layer bawah: trainable=False
- ▶ Ganti **head**: pooling → Dense → Dense(10)

```
1 base = ResNet50(weights='imagenet',  
2     include_top=False,  
3     input_shape=(32,32,3))  
4 base.trainable = False    # freeze  
5 model = Sequential([base,  
6     GlobalAveragePooling2D(),  
7     Dense(128, 'relu'),  
8     Dense(10, 'softmax')])
```

CNN dari Nol vs Transfer Learning

Kapan masing-masing menang?

Aspek	CNN dari Nol	Transfer (ResNet50)
Data dibutuhkan	Banyak	Sedikit
Kecepatan latih	Lambat	Cepat (layer di-freeze)
Data berlabel terbatas	Mudah overfit	Unggul (fitur sudah jadi)
Gambar kecil (32×32)	Bisa bersaing	Tidak selalu unggul

Data terbatas / gambar resolusi tinggi → transfer learning (jalan pintas standar industri).

Act 4

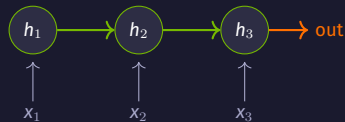
RNN & LSTM

Neural network untuk data berurutan

Data Berurutan & RNN

Neural network yang punya 'memori'

- ▶ **Sequential data:** urutan PENTING (teks, saham, cuaca)
- ▶ "Saya suka kucing" $\neq$ "Kucing suka saya"
- ▶ Dense/CNN proses tiap input independen (tanpa memori)
- ▶ **RNN** ingat langkah sebelumnya via **hidden state** (h)



hidden state diteruskan antar timestep

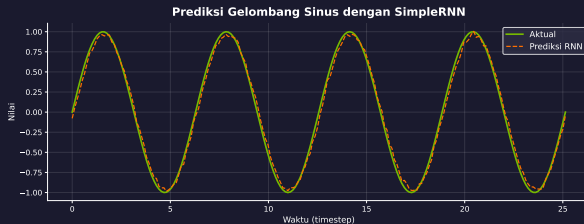
RNN memproses input baru + memori dari langkah sebelumnya (hidden state).

RNN: Prediksi Sinus

Regresi deret waktu dengan SimpleRNN

- ▶ Data: gelombang sinus, pola berulang
- ▶ Input 50 **timesteps** → prediksi langkah ke-51
- ▶ Shape: (samples, timesteps, features)
- ▶ SimpleRNN(32) + Dense(1), loss mse

```
1 SimpleRNN(32, activation='tanh',  
2   input_shape=(50, 1))  
3 Dense(1) # loss='mse'
```



Masalah RNN → LSTM

Mengatasi 'lupa' konteks panjang

- ▶ **Vanishing gradient:** urutan panjang → lupa info awal
- ▶ Seperti bisik berantai: pesan rusak di ujung
- ▶ **LSTM** punya **cell state** (memori jangka panjang)
- ▶ **Gates:** forget (hapus), input (tuliskan), output (baca)

Aspek	SimpleRNN	LSTM
Memori	Pendek	Panjang
Parameter	Sedikit	$\sim 4\times$ lebih
Vanishing grad.	Sering	Teratasi
Kecepatan	Cepat	Lebih lambat

LSTM punya 3 gates + cell state agar info penting bertahan lewat banyak timestep.

LSTM: Sentimen IMDB

Klasifikasi review film positif/negatif

- ▶ Dataset imdb: 50.000 review berlabel
- ▶ **Embedding**: kata $\rightarrow$ vektor 64 dimensi
- ▶ **Padding**: samakan panjang jadi 200 kata
- ▶ Output sigmoid 0-1: positif jika > 0.5



```
1 model = Sequential([
2     Embedding(vocab_size, 64,
3               input_length=200),
4     LSTM(64),
5     Dropout(0.3),
6     Dense(32, activation='relu'),
7     Dense(1, activation='sigmoid'),
8 ])
```

Variasi RNN

SimpleRNN vs LSTM vs GRU

Model	Gates	Kapan dipakai
SimpleRNN	0 (RNN dasar)	Urutan pendek (< 10 langkah)
LSTM	3 (forget/input/output)	Urutan panjang, teks, musik
GRU	2 (update/reset)	Seimbang: cepat & akurat

Evolusi: RNN $\rightarrow$ LSTM $\rightarrow$ GRU $\rightarrow$ Transformer. Pilih sesuai panjang data.

Act 5

Akselerasi GPU NVIDIA

Kenapa deep learning butuh GPU

Kenapa DL Butuh GPU?

CPU vs GPU: serial vs paralel

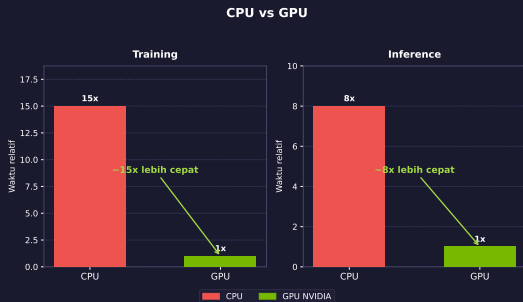
- ▶ **CPU**: sedikit core kuat, kerja **serial**
- ▶ **GPU**: ribuan core sederhana, kerja **paralel**
- ▶ Inti NN = **matrix multiplication**, sangat paralel

Aspek	CPU	GPU NVIDIA
Core	4-16 (kuat)	Ribuan (sederhana)
Cocok	Tugas berurutan	Tugas paralel
Deep Learning	Lambat	Cepat
Contoh	i7, Ryzen	T4, A100, H100

1 guru jenius (CPU) vs 1.000 kalkulator serentak (GPU): DL pilih yang serentak.

Benchmark CPU vs GPU

Matrix multiply, training, inference



- ▶ Uji `tf.matmul` matriks besar di CPU vs GPU
- ▶ Train CNN CIFAR-10, lalu **inference** 1.000 gambar
- ▶ GPU $\sim 10-100\times$ lebih cepat (ilustratif)
- ▶ Makin besar data, makin lebar selisih

```
1 with tf.device('/GPU:0'):  
2     c = tf.matmul(a, b)
```

Mixed Precision Training

Campur FP16 dan FP32

- ▶ **float16** (cepat): forward & backward pass
- ▶ **float32**: bobot master & akumulasi **gradient**
- ▶ **Tensor Cores** NVIDIA percepat operasi float16
- ▶ Hemat memori ~50%, akurasi nyaris tetap

```
1 from tensorflow.keras import \  
2     mixed_precision  
3 mixed_precision.set_global_policy(  
4     'mixed_float16')  
5 # output tetap float32  
6 Dense(10, 'softmax', dtype='float32')
```

Lebih cepat + hemat memori; dipakai melatih GPT, Stable Diffusion, Gemini.

NVIDIA Ecosystem & TensorRT

Dari hardware sampai inference cepat

- ▶ **TensorRT**: optimasi model terlatih untuk inference
- ▶ Trik: layer fusion + precision calibration + auto-tuning
- ▶ Inference 2–5× lebih cepat lewat **TF-TRT** (FP16)

Layer	Komponen
Tools	NGC, NeMo, Triton
Framework	TensorFlow, PyTorch
Library	cuDNN, cuBLAS, TensorRT
Runtime	CUDA
Hardware	T4, A100, H100

Dibaca dari bawah (Hardware) ke atas (Tools) — fondasi semua modul berikutnya.

Act 6

Generative AI

Saat neural network mencipta, bukan hanya mengenali

Diskriminatif vs Generatif

Diskriminatif (Act 1–5)

- ▶ Input data → output **label/angka**
- ▶ “Gambar ini kucing atau anjing?”
- ▶ Belajar **batas** antar kelas

Generatif (Act 6)

- ▶ Noise/konteks → output **data baru**
- ▶ “Buatkan gambar digit yang meyakinkan”
- ▶ Belajar **distribusi** data itu sendiri

Fondasi ChatGPT & Stable Diffusion — dan jembatan kita menuju Module 04 (LLM)

Autoencoder & Latent Space

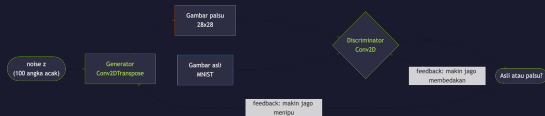
- ▶ Encoder memampatkan 784 pixel $\rightarrow$ **32 angka** (latent); decoder merekonstruksi
- ▶ Decoder pada dasarnya = **generator**: vektor angka $\rightarrow$ gambar
- ▶ Latent space menyimpan **struktur**: titik di antara dua digit juga valid

Interpolasi latent space: 7 $\rightarrow$ 2 (autoencoder, MNIST)



Tapi: sampling *acak* dari latent space AE hasilnya kabur — AE tidak dilatih untuk mencipta. Kita butuh GAN.

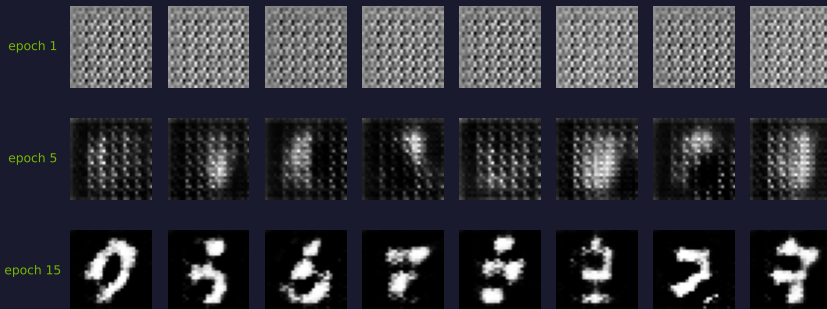
GAN: Generator vs Discriminator



- ▶ **Generator** = pemalsu: noise → gambar palsu
- ▶ **Discriminator** = kurator: asli atau palsu?
- ▶ Dilatih **bergantian** — saling memaksa jadi lebih baik
- ▶ DCGAN: generator Conv2DTranspose, discriminator Conv2D

DCGAN: Digit Lahir dari Noise

DCGAN di MNIST: noise → digit (noise input sama tiap baris)



- ▶ Noise input **sama** di tiap baris — yang berubah hanya kemampuan generator
- ▶ Epoch 1: noise berbintik → epoch 15: mayoritas berbentuk digit
- ▶ Catatan jujur: GAN rewel — *mode collapse* (variasi macet) sering terjadi

Peta Generative AI Modern

Keluarga	Ide inti	Contoh
VAE	latent space diatur supaya bisa di-sampling	generator wajah awal
GAN	generator vs discriminator berlomba	StyleGAN, deepfake
Diffusion	generate = denoise bertahap dari noise murni	Stable Diffusion, DALL-E
Autoregressive	prediksi token berikutnya, berulang	GPT, ChatGPT, Llama

- ▶ Benang merah: semua **belajar distribusi data**, lalu sampling darinya
- ▶ ChatGPT $\approx$ DCGAN kamu: sama-sama generator — bedanya token, bukan pixel

⇒ Module 04: hands-on dengan generator teks (LLM)

Ringkasan Modul 02

Act	Topik	Inti	Praktik
1	Neural Network	Layer, weights, training	Fashion MNIST
2	Aktivasi & Optimizer	Non-linearity, Adam	Activation comparison
3	CNN & Transfer Learning	Convolution, ResNet50	CIFAR-10
4	RNN & LSTM	Hidden state, gates	Sinus, IMDB
5	GPU NVIDIA	Paralelisme, mixed precision	Benchmark, TensorRT
6	Generative AI	Autoencoder & DCGAN	Network yang mencipta, jembatan ke LLM

Lanjutan: Modul 03 – NLP

- ✓ Module 03: **NLP** — memproses bahasa manusia
- ✓ Module 04: **LLM** (Large Language Models)
- ✓ Module 05: **RAG** · Module 06: **NVIDIA ecosystem**



Terima Kasih!

Pertanyaan? Diskusi?

Deep Learning Fundamentals · Modul 02
Navasena Gen-ML Course